

AI Laboratory

Bayesian Networks and Autoregressive Language Models

1. Overview

In the lectures on probability and Bayesian networks, we have seen how probabilistic models can represent uncertainty and how Bayesian networks provide a structured representation of a joint probability distribution.

In this laboratory, we will connect these ideas to a model that is central to modern AI: an **autoregressive language model**.

The central idea is the chain-rule factorisation

$$P(x_1, x_2, \dots, x_T) = P(x_1) \prod_{t=2}^T P(x_t \mid x_1, \dots, x_{t-1}).$$

An autoregressive language model estimates these conditional probabilities and uses them to predict the next word or token.

A simple language model can therefore be viewed as a Bayesian network.

Central question

How can the ideas of Bayesian networks and probabilistic inference help us understand how an autoregressive language model generates text?

2. Learning Objectives

By the end of this laboratory, you should be able to:

1. explain the relationship between the chain rule of probability and autoregressive modelling;
2. represent a simple autoregressive language model as a Bayesian network;
3. construct conditional probability tables from data;
4. use a probabilistic model to predict the next word;
5. generate text by repeatedly sampling from a conditional distribution;
6. use an LLM to assist with the implementation of a probabilistic model;
7. test whether an LLM-generated implementation actually implements the intended probabilistic model;
8. explain why the probabilistic structure of an autoregressive model is important even when the implementation uses a neural network.

3. The AI Laboratory Workflow

The laboratory follows the workflow used throughout this course:

Understand → Design → Ask the LLM → Implement → Test → Reflect

The important point is that the LLM is a **tool for constructing the intelligent system**, not a replacement for understanding the system.

Before asking an LLM to generate code, you should be able to describe:

- what the variables represent;
- what the dependencies between variables are;
- what probability distribution is being estimated;
- what the program should compute.

4. Part I: From Probability to Language

Consider the sentence

`the cat sat on the mat`

Let

$X_1 = \text{the}, \quad X_2 = \text{cat}, \quad X_3 = \text{sat}, \quad X_4 = \text{on}, \quad X_5 = \text{the}, \quad X_6 = \text{mat}.$

The probability of the complete sequence is

$$P(X_1, \dots, X_6).$$

Using the chain rule,

$$\begin{aligned} P(X_1, \dots, X_6) &= P(X_1)P(X_2 \mid X_1)P(X_3 \mid X_1, X_2) \\ &\quad \times P(X_4 \mid X_1, X_2, X_3)P(X_5 \mid X_1, \dots, X_4) \\ &\quad \times P(X_6 \mid X_1, \dots, X_5). \end{aligned}$$

This is an autoregressive decomposition.

Question 1

Why is this decomposition useful for generating text?

Write a short explanation in your lab record.

5. Part II: A Bayesian Network for Text

The full autoregressive model can be represented schematically as

$$X_1 \rightarrow X_2 \rightarrow X_3 \rightarrow \dots \rightarrow X_T$$

if each word depends only on the immediately preceding word.

This is a particularly simple Bayesian network.

For example,

$$X_1 \rightarrow X_2 \rightarrow X_3 \rightarrow X_4$$

represents

$$P(X_1, X_2, X_3, X_4) = P(X_1)P(X_2 | X_1)P(X_3 | X_2)P(X_4 | X_3).$$

This is a **first-order Markov language model**.

It is not a full modern language model.

Instead, it is a deliberately simple model that allows us to see the probabilistic structure clearly.

Question 2

What independence assumption is being made by this network?

Express your answer using probability notation.

6. Part III: Build a Small Language Dataset

For this laboratory, use a small collection of sentences.

You may use the following starting dataset:

```
the cat sat on the mat
the cat sat on the rug
the dog sat on the mat
the dog ran to the park
the cat ran to the park
the dog sat on the rug
```

Convert all text to lower case.

For simplicity, initially treat each word as a token.

Add special tokens

$\langle START \rangle$ $\langle END \rangle$

to mark the beginning and end of each sentence.

For example,

```
<START> the cat sat on the mat <END>
```

7. Part IV: Constructing the Conditional Probability Table

We will begin with a first-order model.

The model estimates

$$P(X_t | X_{t-1}).$$

For example, suppose the training data contains

the cat

three times and

the dog

twice.

Then

$$P(\text{cat} \mid \text{the}) = \frac{3}{5}$$

and

$$P(\text{dog} \mid \text{the}) = \frac{2}{5}.$$

More generally,

$$P(w_j \mid w_i) = \frac{C(w_i, w_j)}{\sum_k C(w_i, w_k)}$$

where $C(w_i, w_j)$ is the number of times that w_j follows w_i in the training data.

Question 3

Construct the conditional probability distribution

$$P(\text{next word} \mid \text{current word})$$

for at least the following words:

the, cat, dog, sat, ran.

Identify any zero-probability transitions.

8. Part V: Ask an LLM to Implement the Model

Now use an LLM to help implement the model.

Do **not** simply ask:

“Write a language model in Python.”

Instead, provide a behavioural specification.

For example:

Suggested prompt

Write a simple Python implementation of a first-order autoregressive language model. The model should:

1. take a list of tokenised sentences as training data;
2. count transitions between consecutive tokens;
3. construct the conditional distribution $P(X_t \mid X_{t-1})$;
4. display the probabilities for a specified previous token;
5. predict the most probable next token;
6. generate a sentence by repeatedly sampling the next token;
7. stop when the <END> token is generated.

Do not use a machine-learning library or a pretrained language model. Use ordinary Python data structures and random sampling.

The purpose of this prompt is important.

You have specified the **behaviour of the probabilistic model** before asking the LLM to implement it.

9. Part VI: Inspect the LLM-Generated Code

Do not immediately assume that the generated program is correct.

Inspect the code.

Answer the following questions.

Question 4

Where in the program are the transition counts stored?

Question 5

Where is

$$P(X_t \mid X_{t-1})$$

computed?

Question 6

How does the program choose the next word?

Is it:

1. always choosing the most probable word, or
2. sampling from the probability distribution?

Explain the difference.

Question 7

What happens if the program encounters a word for which no transition has been observed?

10. Part VII: Test the Probability Model

A probabilistic program should be tested using properties that must hold if the model is correct.

For every current word w ,

$$\sum_v P(v \mid w) = 1.$$

Write a program that checks this property.

For example:

```
for word in probabilities:
    total = sum(probabilities[word].values())
    print(word, total)
```

The totals should be approximately

1.0.

Question 8

If one of the totals is 0.87, what does this tell you about the implementation?

11. Part VIII: Predicting the Next Word

Use your model to calculate the next-word distribution for several contexts.

For example:

$$P(X_{t+1} \mid X_t = \text{the}).$$

Display the resulting distribution.

Then identify

$$\arg \max_w P(w \mid \text{the}).$$

This is the most probable next word according to the model.

Repeat for at least five different preceding words.

Question 9

Are the most probable predictions always the same as the words that you would personally expect?

What does this tell you about the difference between:

a probability model

and

human linguistic expectations?

12. Part IX: Generate Text

Now use the model to generate new sentences.

The generation process is:

$$X_1 \sim P(X_1)$$

followed by

$$X_2 \sim P(X_2 \mid X_1),$$

then

$$X_3 \sim P(X_3 \mid X_2),$$

and so on.

Thus,

sample $\rightarrow$ append token $\rightarrow$ sample again $\rightarrow \dots$

Generate at least 20 sentences.

Save the generated sentences.

13. Part X: Deterministic vs Probabilistic Generation

Modify the program so that it supports two modes.

Mode A: Greedy generation

Always choose

$$\arg \max_w P(w \mid w_{\text{previous}}).$$

Mode B: Sampling

Sample from

$$P(w \mid w_{\text{previous}}).$$

Generate five sentences using each method.

Question 10

Compare the two sets of generated sentences.

Which mode produces more variation?

Why?

14. Part XI: A Second-Order Bayesian Network

Our first model assumes

$$P(X_t \mid X_1, \dots, X_{t-1}) \approx P(X_t \mid X_{t-1}).$$

This is a strong assumption.

Now construct a second-order model:

$$P(X_t \mid X_{t-2}, X_{t-1}).$$

The corresponding Bayesian-network structure is

$$X_{t-2} \longrightarrow X_t \longleftarrow X_{t-1}.$$

For a sequence of four tokens,

$$X_1, X_2, X_3, X_4,$$

the factorisation becomes

$$P(X_1, X_2, X_3, X_4) = P(X_1)P(X_2 \mid X_1)P(X_3 \mid X_1, X_2)P(X_4 \mid X_2, X_3).$$

Question 11

How does the second-order model differ from the first-order model in terms of:

1. the graph structure?
2. the conditional probability table?
3. the amount of context available for prediction?
4. the amount of data needed?

15. Part XII: Use the LLM Again

Now ask the LLM to modify your program to support second-order prediction.

Before accepting the code, state explicitly what should change in the probabilistic model.

A useful prompt might be:

Modify the existing first-order autoregressive model into a second-order model.
The model should estimate

$$P(X_t \mid X_{t-2}, X_{t-1}).$$

Represent the model using counts of observed triples and use these counts to construct conditional probability distributions.

Do not replace the model with a neural network or a pretrained language model.

Again, inspect and test the generated code.

16. Part XIII: Comparing the Two Models

Compare the first-order and second-order models.

Measure at least:

- number of distinct parameters;
- number of zero-probability contexts;
- diversity of generated sentences;
- qualitative coherence of generated sentences.

You should report examples rather than relying only on a numerical score.

Question 12

Why does increasing the amount of context potentially improve prediction?

Why can it simultaneously make the model harder to estimate from limited data?

Relate your answer to the size of the conditional probability table.

17. Part XIV: The Connection to Modern Language Models

A modern autoregressive language model also models

$$P(X_t \mid X_1, \dots, X_{t-1}).$$

The difference is that the conditional distribution is not normally represented by a small hand-built CPT.

Instead, a neural network is trained to estimate the conditional distribution.

Schematically:

$$x_1, \dots, x_{t-1} \longrightarrow \text{neural network} \longrightarrow P(X_t \mid x_1, \dots, x_{t-1})$$

The underlying probabilistic objective remains:

$$P(x_1, \dots, x_T) = \prod_{t=1}^T P(x_t \mid x_1, \dots, x_{t-1})$$

with the first term interpreted as

$$P(x_1 \mid \langle START \rangle).$$

18. Important Conceptual Distinction

The laboratory has deliberately used a very simple model.

A first-order or second-order n -gram model is not comparable in expressive power to a modern large language model.

However, they share an important probabilistic idea:

predict the next token conditional on previous tokens

The difference is primarily in how the conditional distribution is represented and learned.

	Simple BN model	Modern autoregressive LM
Representation	CPTs	Neural network
Context	Fixed window	Large learned context
Parameters	Explicit probabilities	Learned weights
Learning	Counting	Gradient-based training
Generation	Sampling	Sampling

19. Part XV: Reflection on the Role of the LLM

The final part of the laboratory concerns the use of the LLM itself.

Consider the following two approaches.

Approach A

“Write a Python language model for me.”

Approach B

“Implement the following probabilistic model: $P(X_t \mid X_{t-1})$, estimated from transition counts, with sampling-based generation.”

Question 13

Why is Approach B preferable when constructing an intelligent system?

Discuss the importance of:

- specifying the intended behaviour;
- understanding the representation;
- validating the generated implementation;
- testing probabilistic invariants;
- distinguishing implementation from model.

20. Final Question: What Did the Bayesian Network Add?

The final question is deliberately conceptual.

You have now constructed a small autoregressive language model.

Question 14

What did thinking of the language model as a Bayesian network give you?

Discuss at least three of the following:

- a representation of dependencies;
- a factorisation of the joint distribution;
- a way of interpreting conditional probabilities;
- a principled method for generation;
- a way to reason about independence assumptions;
- a way to understand the effect of increasing context;
- a way to test whether an implementation matches its probabilistic specification.

21. Deliverables

Submit:

1. the Python implementation of the first-order model;
2. the Python implementation of the second-order model;
3. the conditional probability tables for selected contexts;
4. examples of generated text;
5. the results of your probability-normalisation tests;
6. your answers to Questions 1–14;
7. a short reflection on how the LLM was used and how you validated its output.

Your reflection should include at least one example of a piece of LLM-generated code that you inspected or corrected.

22. What You Should Take Away

The laboratory illustrates a progression:

Probability $\rightarrow$ Bayesian Network $\rightarrow$ Autoregressive Model $\rightarrow$ Language Generation

The important lesson is not that a simple n -gram model is a modern language model.

Rather, it is that modern autoregressive language models can be understood as sophisticated ways of estimating conditional probability distributions.

The fundamental probabilistic question remains:

$$P(\text{next token} \mid \text{previous tokens})$$

The engineering machinery used to estimate that probability may be very different, but the underlying probabilistic idea remains central.